

Reasoning Under Context Competition: A Controlled Study of Coordination Overhead in LLM Calls

Anonymous Authors

Anonymous Institution

Abstract. Multi-agent and memory-augmented LLM systems often place coordination content, such as shared state, prior discussion, or intermediate summaries, into the same finite context window used for the task itself, creating direct competition between coordination tokens and task-relevant reasoning tokens. We study this tradeoff with the Roundtable Context Window Test (RCWT), a controlled protocol that varies coordination overhead while holding total context budget fixed. Across three commercial models on a context-dependent recall task, performance remains strong through moderate overhead and then degrades sharply at extreme overhead levels; among several simple candidate families, a logistic curve provides the best empirical fit to the observed transition. At a 4,096-token budget, the degradation region appears near 91–94% coordination overhead, although the exact location and steepness are model- and task-dependent. Additional experiments at larger context budgets suggest that this behavior may be better explained by a minimum remaining task-execution context reserve than by a fixed coordination proportion alone, but we view this as a preliminary interpretation that requires validation across additional window sizes and task families. We further find substantial task dependence: self-contained algorithmic tasks show little degradation, contradictory coordination content produces a distinct distraction pattern in one model family, and pack-style DROP probes require a much larger residual task budget than compact reasoning probes. Overall, RCWT provides a controlled methodology for studying context competition in LLM calls and suggests that practical systems should explicitly reserve task-specific execution budget rather than treating coordination context as cost-free.

Keywords: Large language models · Multi-agent systems · Context windows · Reasoning · Benchmarking

1 Introduction

Multi-agent and memory-augmented LLM systems increasingly assemble multiple functional components inside a single model context: agent messages, retrieved memories or documents, summaries of prior context, tool observations, and coordination instructions [18, 19, 22, 9]. This composition is constrained by predefined

context-window limits, which can become binding in settings such as extended conversations, document summarization, and long-horizon reasoning [4]. We study the per-call version of this problem, not full session-level multi-agent dynamics. A single call may contain the task, retrieved memories, summaries of prior turns, intermediate conclusions from other agents, tool outputs, and instructions for how the current model should coordinate with the rest of the system. These components are operationally useful, but they occupy the same finite context window. This motivates studying how task performance changes as non-task coordination content consumes more of the available input budget, especially given evidence that LLMs do not always use long contexts robustly [14, 7].

This paper studies a narrow version of that competition: in a single LLM call with fixed context budget W , what happens as coordination content c increases and the remaining task block $W - c$ shrinks? For convenience, we refer to this fixed-budget tradeoff as *context competition*. For a model M , we define $R_M(c, W, T)$ as the expected response quality on task family T when c tokens of the context window are occupied by coordination content under total budget W . In our experiments, R_M is estimated empirically using task-specific performance metrics such as accuracy. Thus, our object of study is not a universal property of LLMs, but a family of model- and task-dependent response quality curves.

We introduce the Roundtable Context Window Test (RCWT), a controlled protocol that varies coordination overhead while holding total context budget fixed. RCWT is designed to isolate coordination-token pressure from other multi-agent confounds such as turn scheduling, tool failures, memory retrieval policy, and agent topology. The protocol measures a single model call at a time, with fixed task content, fixed coordination content, explicit position control, and binary scoring against precomputed ground truth.

Our central empirical finding is intentionally scoped. On a context-dependent technical-specification recall task across three commercial models, quality remains high through moderate coordination overhead and then degrades sharply at extreme overhead. A logistic curve best summarizes the observed transition among the simple candidate families tested. At $W = 4096$, the transition appears near 91–94% coordination overhead. Additional window-size experiments suggest that the transition is better described by a minimum remaining task budget than by a fixed overhead proportion alone, but this interpretation should be treated as exploratory and task-dependent.

The paper makes four contributions:

1. **A controlled methodology.** RCWT varies coordination proportion with fixed context budget, position control, binary scoring, and multi-provider evaluation.
2. **An empirical characterization.** On the main context-dependent task, all three tested providers show a sharp high-overhead degradation pattern; a logistic curve is the best empirical fit among five simple candidate forms.
3. **A task-dependence analysis.** Self-contained algorithmic tasks do not show the same degradation, while contradictory coordination content produces a distinct model-dependent distraction pattern.

4. **A practical implication.** Coordination content should be budgeted explicitly. For systems operating in task regimes similar to the main RCWT task, preserving a calibrated task-specific residual budget is safer than relying on a fixed coordination proportion.

The claim is not that RCWT discovers a universal context law. The claim is that RCWT gives a reproducible way to measure how task quality changes when coordination content competes with task content inside one LLM call.

2 Related Work

Long-context behavior. Prior work shows that larger context windows do not guarantee uniform use of all positions. The “lost in the middle” result demonstrates that LLMs can underuse relevant information placed away from the beginning or end of a long context [14]. LongBench broadens long-context evaluation across single-document QA, multi-document QA, summarization, few-shot learning, synthetic tasks, and code completion [1]. RULER further expands needle-in-a-haystack style testing with configurable sequence length and task complexity, including multi-hop tracing, aggregation, and QA-style tasks [10]. RCWT asks a different but adjacent question: not only where relevant content is placed or how far a model’s effective context extends, but how quality changes when coordination content consumes increasing fractions of a fixed per-call window.

Attention and systems bottlenecks. Long-context systems face computational and memory pressure from attention and KV-cache growth. Long-context extension methods also show that expanding usable context often requires architectural, training, or attention-pattern changes rather than merely increasing a nominal token limit [4]. Efficient-attention work reduces these costs through IO-aware exact attention or sparse/local/global attention patterns [6, 2, 23]. These methods address how to make long contexts feasible; RCWT measures whether the feasible context is being allocated to coordination or to the task content needed by the current call.

Context compression. Prompt and context compression methods such as LLMingua-2 reduce token count while attempting to preserve task-relevant information [16]. RCWT can be viewed as a complementary measurement tool: if compression reduces coordination tokens without destroying relevant state, it should move a call away from the high-overhead transition region.

Multi-agent coordination overhead. LLM multi-agent systems introduce coordination costs through inter-agent communication, role specification, orchestration, and task verification [3]. Recent scaling studies further show that adding agents can yield diminishing or negative returns when coordination overhead, error amplification, or task structure dominate the benefits of parallelism [13]. However, such systems usually combine several mechanisms at once, including decomposition, debate, voting, tool use, memory, and orchestration. RCWT isolates one component: competition between coordination tokens and task tokens in a single call.

External benchmark probes. GSM8K evaluates grade-school mathematical reasoning [5]; MMLU-Pro evaluates robust multi-task language understanding [20]; and DROP evaluates discrete reasoning over paragraphs [8]. We do not use these as leaderboard benchmarks. Instead, we adapt them into pack-style RCWT stress probes to check whether high-overhead behavior appears outside the original technical-specification task. We do not run RULER in this submission because RULER primarily varies effective sequence length and task complexity, whereas RCWT fixes W and varies the coordination/task allocation inside a single call. A RULER-style RCWT variant is a natural follow-up.

3 Method

3.1 Problem Setup

Let W be a fixed context budget. Let c be the number of tokens allocated to coordination content, and let $p = c/W$ be the coordination proportion. The remaining budget $W - c$ is allocated to task content. RCWT estimates a quality function:

$$R_M(c, W, T) \in [0, 1],$$

where T denotes the task family. Quality is measured against binary ground truth or exact answer extraction, depending on the task.

3.2 RCWT Protocol

The main RCWT task is context-dependent recall over a technical software specification. The coordination block is a realistic specification-like document, and the task block asks for a structured response whose scored facts must be recovered from that document. This design is related to long-context retrieval and recall evaluations, including key-value retrieval and needle-in-a-haystack style probes, but differs in that RCWT varies the fraction of the fixed context budget consumed by coordination content rather than only varying context length or answer position [14, 11]. Two items absent from the reference specification are excluded from the effective score, yielding an 8-item effective score. For example, a scored item may check whether a named configuration option is enabled, deprecated, or subject to a stated constraint in the preceding specification. See Appendix A for a representative example. The main experiment uses $W = 4096$ and coordination proportions $p \in \{0, 25, 50, 75, 90, 92, 94, 96, 98\}\%$. Each cell is run in two position orders: `coord_first`, where coordination content precedes the task, and `reason_first`, where the task precedes coordination content. This order control is motivated by prior evidence that LLM performance can depend strongly on where relevant information appears within a long context [14]. The primary $W = 4096$ suite uses $N = 20$ trials per cell.

The sample size is cost-bounded but targeted at large effects. Pooling both orders gives 40 calls and 320 binary item observations per proportion on the main

8-item effective score. We report Wilson 95% confidence intervals for binomial proportions [21] and two-proportion tests for the main baseline-vs-high-overhead comparison. This design is adequate for the observed 15–34 percentage-point drops at 90% overhead, but not for small-effect claims; those remain exploratory.

3.3 Candidate Curves

For the main context-dependent task, we fit five simple candidate families: logistic, power law, exponential, quadratic, and piecewise linear. These families were chosen as descriptive baselines spanning plausible degradation shapes: a bounded S-shaped threshold, gradual exponential decay, heavy-tailed power-law decay, smooth polynomial curvature, and an explicit breakpoint model. They are not treated as mechanistic proofs. The logistic form is:

$$R_M(c, W, T) = \frac{R_0}{1 + \exp(k(c/W - p_0))},$$

where R_0 is the baseline score, p_0 is the fitted transition location, and k controls transition steepness. We treat this as an empirical summary, not a mechanistic derivation.

3.4 Boundary Tasks

To test whether coordination overhead alone explains degradation, RCWT includes boundary tasks:

- **Self-contained algorithmic task.** The task block contains a Python function, one concrete input, and ten yes/no questions about the execution trace. The correct answers can be computed entirely from the task block; the coordination block is intentionally irrelevant. If performance remains stable as coordination grows, this indicates that overhead is not harmful merely because it adds unrelated text. See Appendix A for a representative example.
- **Context-distraction task.** The task block asks yes/no questions about general data-engineering and product-practice priors. The coordination block is a meeting transcript with salient project-specific decisions that sometimes conflict with those priors, such as choosing an in-house masking layer because a vendor failed a System and Organization Controls (SOC) 2 requirement. This probes semantic distraction: coordination content may hurt by changing which evidence the model treats as salient, not only by consuming tokens. See Appendix A for a representative example.
- **External benchmark probes.** We adapt GSM8K, MMLU-Pro, and DROP into *pack-style* RCWT probes by grouping 10 benchmark items into a single model call. Each pack preserves the original task format at the item level, but the larger combined task block makes coordination overhead materially affect the residual task budget. This design avoids treating isolated one-question prompts as meaningful stress tests of context competition, since such prompts are often too small for high-overhead truncation to affect the task content.

Table 1. Main context-dependent RCWT task at $W = 4096$, pooled over position order. Scores are effective binary accuracy.

Coord. overhead	Task tokens	Gemini	Haiku	GPT
0%	4096	1.000	0.972	1.000
25%	3072	0.960	0.906	0.972
50%	2048	0.944	0.894	0.988
75%	1024	0.894	0.875	0.981
90%	410	0.659	0.666	0.853
92%	328	0.404	0.456	0.600
94%	246	0.012	0.119	0.406
96%	164	0.012	0.088	0.319
98%	82	0.000	0.050	0.337

3.5 Models and Setup

Three commercial models from different providers are tested: Gemini 2.0 Flash, Claude Haiku 4.5, and GPT-4.1-mini. Calls use direct provider APIs rather than a multi-agent framework, because the goal is to isolate the single-call context competition effect.

For double-anonymous review, the replication package is prepared as an anonymized source artifact containing experiment scripts, prompt templates, result summaries, and plotting code. The public repository link will be added after the review process permits deanonymization.

4 Results

4.1 Main Context-Dependent Task

Table 1 shows the pooled main-task scores at $W = 4096$. All three models maintain high scores at low and moderate coordination overhead. Degradation becomes sharp in the 90–98% region.

At 90% overhead, degradation from baseline is 34.1 percentage points for Gemini, 30.6 points for Haiku, and 14.7 points for GPT. The GPT curve is shallower, but not flat: it also degrades significantly in the high-overhead region.

Table 2 summarizes the statistical comparison and fitted transition. Logistic decay has the best AIC among the tested candidate families and high R^2 for all three providers. The fitted transition region is model-dependent but lies within a narrow range for this task at $W = 4096$. The AIC comparison is descriptive model selection over nine sampled proportions, not evidence that the logistic family is a universal law.

(This shouldn’t be here, but in conclusion) –EDIT These results support a practical conclusion: for this class of context-dependent task, coordination overhead can appear harmless through a wide range and then degrade rapidly near the extreme. Average prompt length or nominal context-window size alone is therefore an incomplete safety signal.

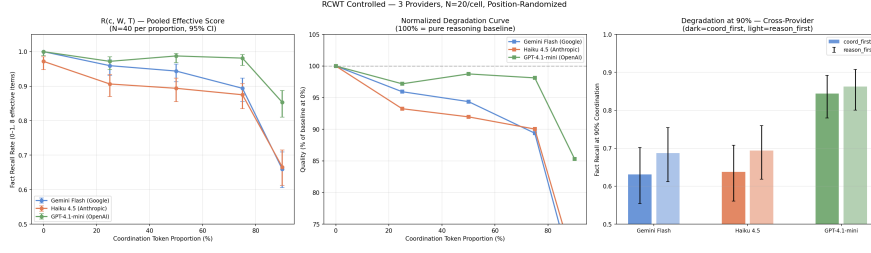


Fig. 1. Cross-provider RCWT scores on the main context-dependent task. The transition appears only at high coordination overhead for this task and budget.

Table 2. Main-task statistical and fit summary at $W = 4096$. Accuracy CIs are Wilson 95% intervals over 320 item-level observations per pooled proportion. p -values are two-proportion z-tests comparing 0% and 90% overhead. AIC reports logistic vs. next-best family.

Model	0% acc. [CI]	90% acc. [CI]	Δ pp	p	p_0	R^2	AIC log./next
Gemini 2.0 Flash	1.000 [0.988,1.000]	0.659 [0.606,0.709]	-34.1	$< 10^{-12}$	0.913	0.988	-24.2 / 0.5
Claude Haiku 4.5	0.972 [0.947,0.985]	0.666 [0.612,0.715]	-30.6	$< 10^{-12}$	0.917	0.990	-27.8 / -1.9
GPT-4.1-mini	1.000 [0.988,1.000]	0.853 [0.810,0.888]	-14.7	$< 10^{-12}$	0.939	0.957	-19.5 / -6.3

4.2 Window Scaling

A fixed-proportion explanation would predict that 90% overhead should have similar effects at different context budgets. The observed pattern does not match that simple account. At $W = 4096$, 90% overhead leaves about 410 task tokens and falls near the degradation region. At $W = 8192$, the same 90% overhead leaves about 819 task tokens and produces much smaller degradation.

We model this with a task-specific residual budget parameter θ . In this interpretation, the transition location follows:

$$p_0(W) = 1 - \frac{\theta}{W}.$$

The point is not that θ is universal; it is an empirical estimate for a given task/model pair. Table 3 summarizes the observed fit across three window sizes for the main task.

Across the $4K$, $8K$, and $16K$ runs, the maximum transition-location error is 0.37 percentage points. This is strong evidence that the fixed-proportion story is incomplete for the main task. However, the interpretation remains bounded: different tasks can have different θ values, and architectural behavior at much larger windows may differ.

A $W = 32768$ extension further supports the direction of the pattern. At that budget, 97.5% overhead leaves about 819 task tokens, and the strongest degradation appears only at 99% overhead, where the remaining task budget is about 328 tokens. The Gemini row in this extension uses Gemini 2.5 Flash rather

Table 3. Window-scaling summary for the main task. Prediction uses θ estimated from the $W = 4096$ fit.

Model	θ	Pred. $p_0(8K)$	Obs. $p_0(8K)$	Obs. $p_0(16K)$
Gemini 2.0 Flash	356	95.7%	96.0%	97.9%
Claude Haiku 4.5	340	95.8%	96.2%	98.1%
GPT-4.1-mini	250	96.9%	97.1%	98.4%

than Gemini 2.0 Flash, so the $32K$ experiment is treated as extension evidence rather than a direct continuation of the original three-model scaling series.

4.3 Task Dependence

RCWT is intended to characterize task regimes, not only model identities. The boundary tasks show that coordination overhead does not have a single universal effect.

Self-Contained Algorithmic Task In the self-contained algorithmic task, the relevant information is in the task block itself. The coordination content is irrelevant. All three providers maintain performance across the tested overhead levels. This suggests that the high-overhead effect in the main task is not caused merely by longer prompts or by the presence of irrelevant text. It appears when coordination content displaces information needed for the task. The aggregate evidence for this boundary task is in `results/task2/`.

Context-Distraction Task The context-distraction task creates a different failure mode. The coordination content contains salient claims that conflict with the correct answers. Haiku is the only tested model that degrades strongly in this setup: its pooled score falls from 0.888 at 0% overhead to 0.670 at 25% and 0.410 at 50%, while GPT remains near 0.88–0.92 and Gemini remains at 0.900. The order split is large for Haiku, so we present this as model-specific exploratory evidence rather than a general law. Detailed aggregate files are included in the supplementary materials; representative prompt fragments appear in Appendix A. This pattern indicates that coordination content can hurt not only by consuming budget, but also by changing which evidence a model treats as salient.

4.4 Cross-Benchmark Generalization

The first one-question GSM8K and MMLU-Pro probes were underpowered for RCWT: their task blocks were short enough that little meaningful truncation occurred at 90% overhead. We therefore use 10-item packs per call and add DROP as a passage-heavy reading-comprehension probe. Table 4 reports the corrected pack probes.

Table 4. Task-dependent residual budget estimates at $W = 4096$. Pack probes are RCWT stress probes, not leaderboard estimates.

Probe	Fitted p_0 range	Approx. reserve	Truncation onset
Task 1 spec recall	0.913–0.939	250–356 tok	main task
GSM8K-pack	0.853–0.893	438–602 tok	90%
MMLU-Pro-pack	0.733–0.755	1003–1094 tok	50% partial
DROP-pack	0.481–0.624	1539–2126 tok	50%

The pack probes are consistent with the RCWT pattern: performance remains near baseline while the full task block fits, then drops when the task block is truncated. The transition location differs by benchmark: DROP shifts earlier because the passage-heavy task block truncates by 50% overhead. These p_0 values are descriptive interpolations over five sampled proportions, not evidence of a universal logistic law. The results should not be cited as model benchmark performance because the prompt format asks for multiple compact answers in one call.

5 Discussion

5.1 Implications

The main engineering implication is simple: coordination context is not free. Prior work on long-context language models shows that performance can degrade as relevant information is displaced within long inputs, and prompt-compression work treats context length as an explicit cost to be managed [14, 12]. Systems that append agent history, retrieved memories, intermediate summaries, and shared state should therefore track the resulting coordination proportion and the residual task budget.

For tasks similar to the main RCWT task, a conservative heuristic is to reserve a few hundred tokens for the task block; the fitted value is roughly 400 tokens at the tested complexity level. This number is not universal; it is a calibration point. The cross-task results show why: the estimated residual budget ranges from a few hundred tokens in the main recall task to roughly 1.5K–2.1K tokens in DROP-style passage reasoning, and the DROP transition is smoother because the task block becomes partially or fully truncated earlier. The safer design pattern is to measure θ for the task family and model, then budget coordination around that value.

Compression and state-structuring methods become useful when they preserve relevant coordination information while reducing token count [12, 16]. The context-distraction results also suggest that structure matters: systems should distinguish factual state, inference, uncertainty, and contradiction rather than mixing them into undifferentiated transcript text. This is consistent with agent-memory systems that separate active context from longer-term or retrieved

memory, and with reflection-based agents that store distilled feedback rather than raw interaction traces [17, 15].

5.2 Threats to Validity

Exploratory design. The main sweep was developed iteratively: early cells suggested a flat region, and cliff cells were added afterward. The fits should be read as empirical characterization and hypothesis generation, not as pre-registered confirmation.

Task coverage. The main logistic result is strongest for one technical-specification recall task. Task 4 and the benchmark packs broaden coverage, but a confirmatory study should pre-register more task families. This matters because RCWT estimates a task-specific residual budget: tasks vary in whether relevant evidence is self-contained, retrieved from context, semantically contradicted by coordination content, or truncated under packing. A broader pre-registered set would separate these regimes more cleanly and reduce the risk that the main logistic pattern reflects the idiosyncrasies of one technical-specification task. **Judge calibration.** The main task uses an automated judge for open-ended response parsing. Cross-vendor re-scoring did not show clear evidence that judges favored outputs from their own model family or provider, but judge-specific strictness remains a limitation.

Tokenization. Providers use different tokenizers. Post-hoc calibration using provider-reported input token counts shows that the cliff-range correction is small relative to the observed model spread, but native tokenization should be preferred in future runs.

Single-call scope. RCWT measures one inference call with controlled context composition. Multi-agent sessions add retrieval policies, memory summarization, turn scheduling, and tool-mediated state changes. RCWT should therefore be used as a local measurement primitive, not as a full session-level theory.

Output length. A possible alternative explanation is that high overhead leaves too little output space, producing shorter responses. Output-length analysis does not support this for Haiku and GPT: responses remain near baseline length while accuracy drops. Gemini shows a partial physical-budget interaction only beyond the quality transition.

6 Conclusion

RCWT provides a controlled way to study context competition in LLM calls. Across three commercial models on a context-dependent recall task, quality remains high through moderate overhead and then degrades sharply near extreme coordination proportions. A logistic curve is the best empirical summary among the candidate families tested, while window-scaling experiments suggest that the transition is better explained by a task-specific execution reserve than by a fixed coordination proportion alone. The result should not be read as a universal threshold of reasoning tokens. It is a measurement framework plus an empirical

warning: coordination content can silently consume the task budget on which the current call depends.

Acknowledgements

We used LLM tools to assist with grammar correction and rephrasing during manuscript preparation. The authors take full responsibility for the content, experiments, analysis, and claims.

A Prompt and Task Examples

The experiments use controlled prompt fragments rather than production logs. Full prompt templates and aggregate files are included in the review artifact.

Main context-dependent task. The coordination block is a technical specification. A representative excerpt states that PostgreSQL LISTEN/NOTIFY has an 8KB payload limit, Redis Pub/Sub handles roughly 100K messages per second but has no persistence, and Redis Streams has persistence with about 5ms latency versus about 1ms for Pub/Sub. The model is asked for a structured technical analysis, and binary scored items check whether these facts are recovered correctly.

Self-contained algorithmic task. The task block gives a Python function that partitions `items=[3, 10, 15, 7, 10, 22, 4]` around `threshold=10`. Representative scored questions ask whether the sum is 71, whether indices 2 and 5 are above threshold, and whether the maximum above-threshold value is 22. The answers are fully determined by the code and input, so the coordination block is irrelevant.

Context-distraction task. The coordination block is a controlled meeting transcript for an invented data-platform project. It states that Kafka Streams was selected after load tests, 180-day retention was chosen, and an in-house masking layer was selected because a vendor failed a SOC 2 requirement. The task block asks broader yes/no questions such as whether building a data-masking layer in-house is generally more cost-effective than buying a third-party SaaS solution. The correct answer is NO, even though the project-specific transcript says to build in-house; this creates a salient contradiction between coordination content and the target prior.

References

1. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: Longbench: A bilingual, multitask benchmark for long context understanding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (2024), <https://arxiv.org/abs/2308.14508>
2. Beltagy, I., Peters, M.E., Cohan, A.: Longformer: The long-document transformer (2020), <https://arxiv.org/abs/2004.05150>

3. Cemri, M., Pan, M.Z., Yang, S., Agrawal, L.A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J.E., Stoica, I.: Why do multi-agent LLM systems fail? (2025), <https://arxiv.org/abs/2503.13657>
4. Chen, Y., Qian, S., Tang, H., Lai, X., Liu, Z., Han, S., Jia, J.: LongloRA: Efficient fine-tuning of long-context large language models. In: The Twelfth International Conference on Learning Representations (2024), <https://openreview.net/forum?id=6PmJoRfdak>
5. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training verifiers to solve math word problems (2021), <https://arxiv.org/abs/2110.14168>
6. Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Ré, C.: Flashattention: Fast and memory-efficient exact attention with io-awareness. In: Advances in Neural Information Processing Systems (2022), <https://arxiv.org/abs/2205.14135>
7. Du, Y., Tian, M., Ronanki, S., Rongali, S., Bodapati, S.B., Galstyan, A., Wells, A., Schwartz, R., Huerta, E.A., Peng, H.: Context length alone hurts LLM performance despite perfect retrieval. In: Findings of the Association for Computational Linguistics: EMNLP 2025. pp. 23281–23298. Association for Computational Linguistics, Suzhou, China (Nov 2025). <https://doi.org/10.18653/v1/2025.findings-emnlp.1264>, <https://aclanthology.org/2025.findings-emnlp.1264/>
8. Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., Gardner, M.: DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 2368–2378. Association for Computational Linguistics, Minneapolis, Minnesota (2019). <https://doi.org/10.18653/v1/N19-1246>, <https://aclanthology.org/N19-1246/>
9. Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X.: Large language model based multi-agents: A survey of progress and challenges (2024), <https://arxiv.org/abs/2402.01680>
10. Hsieh, C.P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., Ginsburg, B.: Ruler: What’s the real context size of your long-context language models? In: Proceedings of the First Conference on Language Modeling (2024), <https://arxiv.org/abs/2404.06654>
11. Ivgi, M., Shaham, U., Berant, J.: Efficient long-text understanding with short-text models. Transactions of the Association for Computational Linguistics **11**, 284–299 (2023). https://doi.org/10.1162/tacl_a_00547, <https://aclanthology.org/2023.tacl-1.17/>
12. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 13358–13376. Association for Computational Linguistics, Singapore (Dec 2023). <https://doi.org/10.18653/v1/2023.emnlp-main.825>, <https://aclanthology.org/2023.emnlp-main.825/>
13. Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A.A., Yan, Y., Zhang, Z., Zhuang, Y., Liu, Y., Malhotra, M., Liang, P.P., Park, H.W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., Liu, X.: Towards a science of scaling agent systems (2026), <https://arxiv.org/abs/2512.08296>
14. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. Transactions of

- the Association for Computational Linguistics **12**, 157–173 (2024), <https://arxiv.org/abs/2307.03172>
15. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as operating systems (2024), <https://arxiv.org/abs/2310.08560>
 16. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.Y., Zhao, H.V., Qiu, L., Zhang, D.: LlmLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In: Findings of the Association for Computational Linguistics: ACL 2024 (2024), <https://arxiv.org/abs/2403.12968>
 17. Park, J.S., O’Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Generative agents: Interactive simulacra of human behavior (2023), <https://arxiv.org/abs/2304.03442>
 18. Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K., Shoham, Y.: In-context retrieval-augmented language models. Transactions of the Association for Computational Linguistics **11**, 1316–1331 (2023). https://doi.org/10.1162/tacl_a_00605, <https://aclanthology.org/2023.tacl-1.75/>
 19. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems. vol. 36, pp. 68539–68551. Curran Associates, Inc. (2023), https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf
 20. Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., Chen, W.: Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track (2024), <https://arxiv.org/abs/2406.01574>
 21. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association **22**(158), 209–212 (1927). <https://doi.org/10.1080/01621459.1927.10502953>
 22. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: Autogen: Enabling next-gen LLM applications via multi-agent conversations. In: First Conference on Language Modeling (2024), <https://openreview.net/forum?id=BAakY1hNKS>
 23. Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., Ahmed, A.: Big bird: Transformers for longer sequences. In: Advances in Neural Information Processing Systems (2020), <https://arxiv.org/abs/2007.14062>